Flattening a type family over a higher-inductive homotopy coequalizer

The Univalent Foundations Program

hott-book:578b85cc8d586b1677ec4335148adeb443057d24; CC BY-SA 3.0 Unported

Editorial note. The actual finalProof of thm:flattening2025–2039 is retained with every preceding local constructor/induction/recursor lemma1853–2021. The task does not present that final short paragraph without its core.

Editorial note. The source explicitly avoids proving an unnecessary full dependent path-computation rule at1922–1927 and proves the nondependent rule actually needed. The selected author argument is retained as such; no missing alternate rule is supplied.

Editorial note. This excerpt retainsCC BY-SA3.0Unported attribution and share-alike terms.

Allowed standard prerequisites (editorial).

- Basic univalent type-theoretic foundations. Univalence, function extensionality, dependent sums, path/transport induction and the defining induction/recursion principles of the homotopy coequalizer. The flattening constructors, induced recursor and path-computation rule are retained author obligations.

Source reference. Original section 8: focused-seeds/AGP/sources/hott-book-578b85cc/extracted/book-578b85cc8d586b1677ec4335148adeb443057d24/main.labelnumbers.first-edition, source lines 167–167. Original numbered reference to allowed path/transport foundations or an unselected illustrative discussion; no pointer replaces the retained flattening construction.

Source reference. Original lemma 2.10.5: focused-seeds/AGP/sources/hott-book-578b85cc/extracted/book-578b85cc8d586b1677ec4335148adeb443057d24/main.labelnumbers.first-edition, source lines 1897–1897. Original numbered reference to allowed path/transport foundations or an unselected illustrative discussion; no pointer replaces the retained flattening construction.

Source reference. Original section 2.12: focused-seeds/AGP/sources/hott-book-578b85cc/extracted/book-578b85cc8d586b1677ec4335148adeb443057d24/main.labelnumbers.first-edition, source lines 1625–1625. Original numbered reference to allowed path/transport foundations or an unselected illustrative discussion; no pointer replaces the retained flattening construction.

Source reference. Original section 2.13: focused-seeds/AGP/sources/hott-book-578b85cc/extracted/book-578b85cc8d586b1677ec4335148adeb443057d24/main.labelnumbers.first-edition, source lines 1626–1626. Original numbered reference to allowed path/transport

foundations or an unselected illustrative discussion; no pointer replaces the retained flattening construction.

Source reference. Original lemma 2.9.7: focused-seeds/AGP/sources/hott-book-578b85cc/extracted/book-578b85cc8d586b1677ec4335148adeb443057d24/main.labelnumbers.first-edition, source lines 1765–1765. Original numbered reference to allowed path/transport foundations or an unselected illustrative discussion; no pointer replaces the retained flattening construction.

Source reference. Original lemma 2.3.10: focused-seeds/AGP/sources/hott-book-578b85cc/extracted/book-578b85cc8d586b1677ec4335148adeb443057d24/main.labelnumbers.first-edition, source lines 1894–1894. Original numbered reference to allowed path/transport foundations or an unselected illustrative discussion; no pointer replaces the retained flattening construction.

Source reference. Original lemma 2.9.6: focused-seeds/AGP/sources/hott-book-578b85cc/extracted/book-578b85cc8d586b1677ec4335148adeb443057d24/main.labelnumbers.first-edition, source lines 1764–1764. Original numbered reference to allowed path/transport foundations or an unselected illustrative discussion; no pointer replaces the retained flattening construction.

Source reference. Original lemma 2.3.5: focused-seeds/AGP/sources/hott-book-578b85cc/extracted/book-578b85cc8d586b1677ec4335148adeb443057d24/main.labelnumbers.first-edition, source lines 1902–1902. Original numbered reference to allowed path/transport foundations or an unselected illustrative discussion; no pointer replaces the retained flattening construction.

Source reference. Original equation 2.9.4: focused-seeds/AGP/sources/hott-book-578b85cc/extracted/book-578b85cc8d586b1677ec4335148adeb443057d24/main.labelnumbers.first-edition, source lines 1324–1324. Original numbered reference to allowed path/transport foundations or an unselected illustrative discussion; no pointer replaces the retained flattening construction.

6.12 The flattening lemma

As we will see in original section 8, amazing things happen when we combine higher inductive types with univalence. The principal way this comes about is that if W is a higher inductive type and $\mathcal{U}$ is a type universe, then we can define a type family $P : W \to \mathcal{U}$ by using the recursion principle for W. When we come to the clauses of the recursion principle dealing with the path constructors of W, we will need to supply paths in $\mathcal{U}$, and this is where univalence comes in.

For example, suppose we have a type X and a self-equivalence $e : X \simeq X$. Then we can define a type family $P : \mathbb{S}^1 \to \mathcal{U}$ by using $\mathbb{S}^1$-recursion:

$$P(\mathsf{base}) :\equiv X \qquad \text{and} \qquad P(\mathsf{loop}) \coloneqq \mathsf{ua}(e).$$

The type X thus appears as the fiber $P(\mathsf{base})$ of P over the basepoint. The self-equivalence e is a little more hidden in P, but the following lemma says that it can be extracted by transporting along loop.

Lemma 6.12.1. *Given $B : A \to \mathcal{U}$ and $x, y : A$, with a path $p : x = y$ and an equivalence $e : B(x) \simeq B(y)$ such that $B(p) = \mathsf{ua}(e)$, then for any $u : B(x)$ we have*

$$\mathsf{transport}^B(p, u) = e(u).$$

Proof. Applying original lemma 2.10.5, we have

$$\begin{aligned} \mathsf{transport}^B(p, u) &= \mathsf{idtoeqv}(B(p))(u) \\ &= \mathsf{idtoeqv}(\mathsf{ua}(e))(u) \\ &= e(u). \end{aligned}$$

□

We have seen type families defined by recursion before: in original sections 2.12 and 2.13 we used them to characterize the identity types of (ordinary) inductive types. In original section 8, we will use similar ideas to calculate homotopy groups of higher inductive types.

In this section, we describe a general lemma about type families of this sort which will be useful later on. We call it the **flattening lemma**: it says that if $P : W \to \mathcal{U}$ is defined recursively as above, then its total space $\sum_{(x:W)} P(x)$ is equivalent to a "flattened" higher inductive type, whose constructors may be deduced from those of W and the definition of P. (From a category-theoretic point of view, $\sum_{(x:W)} P(x)$ is the "Grothendieck construction" of P, and the flattening lemma expresses its universal property as a "lax colimit". Although because types in homotopy type theory (like W) correspond categorically to ∞-groupoids (since all paths are invertible), in this case the lax colimit is the same as a pseudo colimit.)

We prove here one general case of the flattening lemma, which directly implies many particular cases and suggests the method to prove others. Suppose we have $A, B : \mathcal{U}$ and $f, g : B \to A$, and that the higher inductive type W is generated by

- $\mathsf{c} : A \to W$ and
- $\mathsf{p} : \prod_{(b:B)} (\mathsf{c}(f(b)) =_W \mathsf{c}(g(b)))$.

Thus, W is the **(homotopy) coequalizer** of f and g. Using binary sums (coproducts) and dependent sums (Σ-types), a lot of interesting nonrecursive higher inductive types can be represented in this form. All point constructors have to be bundled in the type A and all path constructors in the type B. For instance:

- The circle $\mathbb{S}^1$ can be represented by taking $A :\equiv \mathbf{1}$ and $B :\equiv \mathbf{1}$, with f and g the identity.
- The pushout of $j : X \to Y$ and $k : X \to Z$ can be represented by taking $A :\equiv Y + Z$ and $B :\equiv X$, with $f :\equiv \mathsf{inl} \circ j$ and $g :\equiv \mathsf{inr} \circ k$.

Now suppose in addition that

- $C : A \to \mathcal{U}$ is a family of types over A, and
- $D : \prod_{(b:B)} C(f(b)) \simeq C(g(b))$ is a family of equivalences over B.

Define a type family $P : W \to \mathcal{U}$ recursively by

$$P(\mathsf{c}(a)) :\equiv C(a)$$
$$P(\mathsf{p}(b)) \coloneqq \mathsf{ua}(D(b)).$$

Let $\widetilde{W}$ be the higher inductive type generated by

- $\widetilde{\mathsf{c}} : \prod_{(a:A)} C(a) \to \widetilde{W}$ and
- $\widetilde{\mathsf{p}} : \prod_{(b:B)} \prod_{(y:C(f(b)))} (\widetilde{\mathsf{c}}(f(b), y) =_{\widetilde{W}} \widetilde{\mathsf{c}}(g(b), D(b)(y)))$.

The flattening lemma is:

Lemma 6.12.2 (Flattening lemma)**.** *In the above situation, we have*

$$\Big(\sum_{x:W} P(x)\Big) \simeq \widetilde{W}.$$

As remarked above, this equivalence can be seen as expressing the universal property of $\sum_{(x:W)} P(x)$ as a "lax colimit" of P over W. It can also be seen as part of the *stability and descent* property of colimits, which characterizes higher toposes.

The proof of Lemma 6.12.2 occupies the rest of this section. It is somewhat technical and can be skipped on a first reading. But it is also a good example of "proof-relevant mathematics", so we recommend it on a second reading.

The idea is to show that $\sum_{(x:W)} P(x)$ has the same universal property as $\widetilde{W}$. We begin by showing that it comes with analogues of the constructors $\widetilde{\mathsf{c}}$ and $\widetilde{\mathsf{p}}$.

Lemma 6.12.3. *There are functions*

- $\widetilde{\mathsf{c}}' : \prod_{(a:A)} C(a) \to \sum_{(x:W)} P(x)$ *and*
- $\widetilde{\mathsf{p}}' : \prod_{(b:B)} \prod_{(y:C(f(b)))} \Big(\widetilde{\mathsf{c}}'(f(b), y) =_{\sum_{(w:W)} P(w)} \widetilde{\mathsf{c}}'(g(b), D(b)(y))\Big)$.

Proof. The first is easy; define $\widetilde{\mathsf{c}}'(a, x) :\equiv (\mathsf{c}(a), x)$ and note that by definition $P(\mathsf{c}(a)) \equiv C(a)$. For the second, suppose given $b : B$ and $y : C(f(b))$; we must give an equality

$$(\mathsf{c}(f(b)), y) = (\mathsf{c}(g(b)), D(b)(y)).$$

Since we have $\mathsf{p}(b) : \mathsf{c}(f(b)) = \mathsf{c}(g(b))$, by equalities in Σ-types it suffices to give an equality $\mathsf{p}(b)_*(y) = D(b)(y)$. But this follows from Lemma 6.12.1, using the definition of P. □

Now the following lemma says to define a section of a type family over $\sum_{(w:W)} P(w)$, it suffices to give analogous data as in the case of $\widetilde{W}$.

Lemma 6.12.4. *Suppose $Q : \big(\sum_{(x:W)} P(x)\big) \to \mathcal{U}$ is a type family and that we have*

- $c : \prod_{(a:A)} \prod_{(x:C(a))} Q(\widetilde{\mathsf{c}}'(a, x))$ *and*
- $p : \prod_{(b:B)} \prod_{(y:C(f(b)))} \Big(\widetilde{\mathsf{p}}'(b, y)_*(c(f(b), y)) = c(g(b), D(b)(y))\Big)$.

Then there exists $k : \prod_{(z:\sum_{(w:W)} P(w))} Q(z)$ such that $k(\widetilde{\mathsf{c}}'(a, x)) \equiv c(a, x)$.

Proof. Suppose given $w : W$ and $x : P(w)$; we must produce an element $k(w, x) : Q(w, x)$. By induction on w, it suffices to consider two cases. When $w \equiv \mathsf{c}(a)$, then we have $x : C(a)$, and so $c(a, x) : Q(\mathsf{c}(a), x)$ as desired. (This part of the definition also ensures that the stated computational rule holds.)

Now we must show that this definition is preserved by transporting along $\mathsf{p}(b)$ for any $b : B$. Since what we are defining, for all $w : W$, is a function of type $\prod_{(x:P(w))} Q(w, x)$, by original lemma 2.9.7 it suffices to show that for any $y : C(f(b))$, we have

$$\mathsf{transport}^Q(\mathsf{pair}^=(\mathsf{p}(b), \mathsf{refl}_{\mathsf{p}(b)_*(y)}), c(f(b), y)) = c(g(b), \mathsf{p}(b)_*(y)).$$

Let $q : \mathsf{p}(b)_*(y) = D(b)(y)$ be the path obtained from Lemma 6.12.1. Then we have

$$\begin{aligned} c(g(b), \mathsf{p}(b)_*(y)) &= \mathsf{transport}^{x \mapsto Q(\widetilde{\mathsf{c}}'(g(b),x))}(q^{-1}, c(g(b), D(b)(y))) \\ &\qquad \text{(by } \mathsf{apd}_{x \mapsto c(g(b),x)}(q^{-1})^{-1}\text{)} \\ &= \mathsf{transport}^Q(\mathsf{ap}_{x \mapsto \widetilde{\mathsf{c}}'(g(b),x)}(q^{-1}), c(g(b), D(b)(y))). \\ &\qquad \text{(by original lemma 2.3.10)} \end{aligned}$$

Thus, it suffices to show

$$\mathsf{transport}^Q\Big(\mathsf{pair}^=(\mathsf{p}(b), \mathsf{refl}_{\mathsf{p}(b)_*(y)}),\ c(f(b), y)\Big) = \mathsf{transport}^Q\Big(\mathsf{ap}_{x \mapsto \widetilde{\mathsf{c}}'(g(b),x)}(q^{-1}),\ c(g(b), D(b)(y))\Big).$$

Moving the right-hand transport to the other side, and combining two transports, this is equivalent to

$$\mathsf{transport}^Q\Big(\mathsf{pair}^=(\mathsf{p}(b), \mathsf{refl}_{\mathsf{p}(b)_*(y)}) \cdot \mathsf{ap}_{x \mapsto \widetilde{\mathsf{c}}'(g(b),x)}(q),\ c(f(b), y)\Big) = c(g(b), D(b)(y)).$$

However, we have

$$\begin{aligned} &\mathsf{pair}^=(\mathsf{p}(b), \mathsf{refl}_{\mathsf{p}(b)_*(y)}) \cdot \mathsf{ap}_{x \mapsto \widetilde{\mathsf{c}}'(g(b),x)}(q) = \\ &\qquad \mathsf{pair}^=(\mathsf{p}(b), \mathsf{refl}_{\mathsf{p}(b)_*(y)}) \cdot \mathsf{pair}^=(\mathsf{refl}_{\mathsf{c}(g(b))}, q) = \mathsf{pair}^=(\mathsf{p}(b), q) = \widetilde{\mathsf{p}}'(b, y) \end{aligned}$$

so the construction is completed by the assumption $p(b, y)$ of type

$$\mathsf{transport}^Q(\widetilde{\mathsf{p}}'(b, y), c(f(b), y)) = c(g(b), D(b)(y)). \qquad \square$$

Lemma 6.12.4 *almost* gives $\sum_{(w:W)} P(w)$ the same induction principle as $\widetilde{W}$. The missing bit is the equality $\mathsf{apd}_k(\widetilde{\mathsf{p}}'(b, y)) = p(b, y)$. In order to prove this, we would need to analyze the proof of Lemma 6.12.4, which of course is the definition of k.

It should be possible to do this, but it turns out that we only need the computation rule for the non-dependent recursion principle. Thus, we now give a somewhat simpler direct construction of the recursor, and a proof of its computation rule.

Lemma 6.12.5. *Suppose Q is a type and that we have*

- $c : \prod_{(a:A)} C(a) \to Q$ *and*

- $p : \prod_{(b:B)} \prod_{(y:C(f(b)))} \Big(c(f(b), y) =_Q c(g(b), D(b)(y))\Big)$.

Then there exists $k : \big(\sum_{(w:W)} P(w)\big) \to Q$ *such that* $k(\widetilde{\mathsf{c}}'(a, x)) \equiv c(a, x)$.

Proof. As in Lemma 6.12.4, we define $k(w, x)$ by induction on $w : W$. When $w \equiv \mathsf{c}(a)$, we define $k(\mathsf{c}(a), x) :\equiv c(a, x)$. Now by original lemma 2.9.6, it suffices to consider, for $b : B$ and $y : C(f(b))$, the composite path

$$\mathsf{transport}^{x \mapsto Q}(\mathsf{p}(b), c(f(b), y)) = c(g(b), \mathsf{transport}^{P}(\mathsf{p}(b), y)) \tag{6.12.6}$$

defined as the composition

$$\begin{aligned}
\mathsf{transport}^{x \mapsto Q}(\mathsf{p}(b), c(f(b), y)) &= c(f(b), y) && \text{(by original lemma 2.3.5)} \\
&= c(g(b), D(b)(y)) && \text{(by } p(b, y)\text{)} \\
&= c(g(b), \mathsf{transport}^{P}(\mathsf{p}(b), y)). && \text{(by Lemma 6.12.1)}
\end{aligned}$$

The computation rule $k(\widetilde{\mathsf{c}}'(a, x)) \equiv c(a, x)$ follows by definition, as before. □

For the second computation rule, we need the following lemma.

Lemma 6.12.7. *Let* $Y : X \to \mathcal{U}$ *be a type family and let* $k : (\sum_{(x:X)} Y(x)) \to Z$ *be defined componentwise by* $k(x, y) :\equiv d(x)(y)$ *for a curried function* $d : \prod_{(x:X)} Y(x) \to Z$. *Then for any* $s : x_1 =_X x_2$ *and any* $y_1 : Y(x_1)$ *and* $y_2 : Y(x_2)$ *with a path* $r : s_*(y_1) = y_2$, *the path*

$$\mathsf{ap}_k(\mathsf{pair}^{=}(s, r)) : k(x_1, y_1) = k(x_2, y_2)$$

is equal to the composite

$$\begin{aligned}
k(x_1, y_1) &\equiv d(x_1)(y_1) \\
&= \mathsf{transport}^{x \mapsto Z}(s, d(x_1)(y_1)) && \text{(by (original lemma 2.3.5)}^{-1}\text{)} \\
&= \mathsf{transport}^{x \mapsto Z}(s, d(x_1)({s^{-1}}_*(s_*(y_1)))) \\
&= \big(\mathsf{transport}^{x \mapsto (Y(x) \to Z)}(s, d(x_1))\big)(s_*(y_1)) && \text{(by (2.9.4))} \\
&= d(x_2)(s_*(y_1)) && \text{(by } \mathsf{happly}(\mathsf{apd}_d(s))(s_*(y_1))\text{)} \\
&= d(x_2)(y_2) && \text{(by } \mathsf{ap}_{d(x_2)}(r)\text{)} \\
&\equiv k(x_2, y_2).
\end{aligned}$$

Proof. After path induction on s and r, both equalities reduce to reflexivities. □

At first it may seem surprising that Lemma 6.12.7 has such a complicated statement, while it can be proven so simply. The reason for the complication is to ensure that the statement is well-typed: $\mathsf{ap}_k(\mathsf{pair}^{=}(s, r))$ and the composite path it is claimed to be equal to must both have the same start and end points. Once we have managed this, the proof is easy by path induction.

Lemma 6.12.8. *In the situation of Lemma 6.12.5, we have* $\mathsf{ap}_k(\widetilde{\mathsf{p}}'(b, y)) = p(b, y)$.

Proof. Recall that $\widetilde{\mathsf{p}}'(b,y) :\equiv \mathsf{pair}^{=}(\mathsf{p}(b), q)$ where $q : \mathsf{p}(b)_*(y) = D(b)(y)$ comes from Lemma 6.12.1. Thus, since k is defined componentwise, we may compute $\mathsf{ap}_k(\widetilde{\mathsf{p}}'(b,y))$ by Lemma 6.12.7, with

$$\begin{aligned} x_1 &:\equiv \mathsf{c}(f(b)) & y_1 &:\equiv y \\ x_2 &:\equiv \mathsf{c}(g(b)) & y_2 &:\equiv D(b)(y) \\ s &:\equiv \mathsf{p}(b) & r &:\equiv q. \end{aligned}$$

The curried function $d : \prod_{(w:W)} P(w) \to Q$ was defined by induction on $w : W$; to apply Lemma 6.12.7 we need to understand $\mathsf{ap}_{d(x_2)}(r)$ and $\mathsf{happly}(\mathsf{apd}_d(s), s_*(y_1))$.

For the first, since $d(\mathsf{c}(a), x) \equiv c(a, x)$, we have

$$\mathsf{ap}_{d(x_2)}(r) \equiv \mathsf{ap}_{c(g(b),-)}(q).$$

For the second, the computation rule for the induction principle of W tells us that $\mathsf{apd}_d(\mathsf{p}(b))$ is equal to the composite (6.12.6), passed across the equivalence of original lemma 2.9.6. Thus, the computation rule given in original lemma 2.9.6 implies that $\mathsf{happly}(\mathsf{apd}_d(\mathsf{p}(b)), \mathsf{p}(b)_*(y))$ is equal to the composite

$$\begin{aligned} \big(\mathsf{p}(b)_*(c(f(b),-))\big)(\mathsf{p}(b)_*(y)) &= \mathsf{p}(b)_*\Big(c(f(b), \mathsf{p}(b)^{-1}{}_*(\mathsf{p}(b)_*(y)))\Big) && \text{(by (2.9.4))} \\ &= \mathsf{p}(b)_*(c(f(b),y)) \\ &= c(f(b),y) && \text{(by original lemma 2.3.5)} \\ &= c(g(b), D(b)(y)) && \text{(by } p(b,y)\text{)} \\ &= c(g(b), \mathsf{p}(b)_*(y)). && (\text{by } \mathsf{ap}_{c(g(b),-)}(q)^{-1}) \end{aligned}$$

Finally, substituting these values of $\mathsf{ap}_{d(x_2)}(r)$ and $\mathsf{happly}(\mathsf{apd}_d(s), s_*(y_1))$ into Lemma 6.12.7, we see that all the paths cancel out in pairs, leaving only $p(b,y)$. □

Now we are finally ready to prove the flattening lemma.

Proof of Lemma 6.12.2. We define $h : \widetilde{W} \to \sum_{(w:W)} P(w)$ by using the recursion principle for $\widetilde{W}$, with $\widetilde{\mathsf{c}}'$ and $\widetilde{\mathsf{p}}'$ as input data. Similarly, we define $k : (\sum_{(w:W)} P(w)) \to \widetilde{W}$ by using the recursion principle of Lemma 6.12.5, with $\widetilde{\mathsf{c}}$ and $\widetilde{\mathsf{p}}$ as input data.

On the one hand, we must show that for any $z : \widetilde{W}$, we have $k(h(z)) = z$. By induction on z, it suffices to consider the two constructors of $\widetilde{W}$. But we have

$$k(h(\widetilde{\mathsf{c}}(a,x))) \equiv k(\widetilde{\mathsf{c}}'(a,x)) \equiv \widetilde{\mathsf{c}}(a,x)$$

by definition, while similarly

$$k(h(\widetilde{\mathsf{p}}(b,y))) = k(\widetilde{\mathsf{p}}'(b,y)) = \widetilde{\mathsf{p}}(b,y)$$

using the propositional computation rule for $\widetilde{W}$ and Lemma 6.12.8.

On the other hand, we must show that for any $z : \sum_{(w:W)} P(w)$, we have $h(k(z)) = z$. But this is essentially identical, using Lemma 6.12.4 for "induction on $\sum_{(w:W)} P(w)$" and the same computation rules. □